Propositional MKNF and MBNF, mechanized

Samuele Manclossi
MSc. student in Computer Science
Matriculation nr. 84275A
Computer Science Department
University of Milan
samuele.manclossi@studenti.unimi.it

***Abstract*— Currently, the logics of minimal knowledge/belief and negation as failure (MKNF and MBNF respectively) are two well-established formalisms for nonmonotonic reasoning [1, 2], with applications of hybrid MKNF to the semantic web [3] and formal modelling of legal and normative systems [4]. This document reports over one important step for these logics: the implementation of their propositional syntax, semantics, and reasoning algorithms inside Rocq, a mature proof assistant [5], in order to prove the correctness of the reasoning techniques developed through the years. The consequences of this development are threefold: first, it provides certification of the pen-and-paper proofs formalised here; second, it guarantees formally proved correct implementations of the reasoning algorithms presented here; third, it provides useful resources for practitioners whose intent is to learn these formalisms.**

I. INTRODUCTION AND BACKGROUND

This section introduces the most relevant concepts useful to understand the whole report.

A. Propositional MKNF

The propositional subset of the logic of Minimal Knowledge with Negation as Failure (MKNF for short) is a modal logic introduced by Lifschitz [1]. It uses all the classical operators, plus two modal ones, K and *not*. The truth value of a proposition is determined with respect to a triple (I, S^k, S^n) where I is a classical interpretation, S^k, S^n are subsets of interpretations. In particular, KF holds when F holds in every world of S^k, and *not* F holds when there exists a world in S^n in which $\neg F$ holds. In particular, we say that S is a model of a theory T if and only if every $I \in S$ satisfies T in (I, S, S) and (this is the knowledge minimality condition) there is no $S' \supset S$ which satisfies T for all $I \in S'$ in (I, S', S) [1, Sec. 4].

Whether empty models are accepted depends on the convention, and we provide both implementations [1, Sec. 4]. The semantic link between MKNF and MBNF is built over non-empty models [2, Def. 2.1 and Sec. 5].

B. Propositional MBNF

The logic of Minimal Belief with Negation as Failure (MBNF for short) is very similar to MKNF in its syntax, with B and *not* as modal operators [2, Sec. 2]. The conditions for (I, M, M) to be a model of T are that M is non-empty, that (I, M, M) satisfies T, and (this is the belief minimality condition) that there is no pair of an interpretation J and a cluster $M' \supset M$ such that (J, M', M) satisfies T. This minimality check changes both the belief worlds and the initial one, while keeping the assumptions fixed [2, Def. 2.1].

C. Semantic link and subjectivization

The two logics, while distinct and with different semantics, are tightly connected. Consider a non-empty MKNF model M of T. This happens if and only if (I, M, M) is an MBNF model for the theory $\{B\tau(F)\}_{F \in T}$, for every initial interpretation I, where τ is a syntactic operator that replaces each occurrence of K with an occurrence of B [2, Sec. 5, p. 295].

D. Logical consequence and reasoning over MKNF/MBNF

The knowledge representation and reasoning field focuses on syntax and semantics as well as reasoning. Considering the latter, the fact that the entailment relationship for MKNF can be established by subjectivization and the usage of MBNF algorithms makes it even more relevant to implement and certify them. The two procedures considered here are the exhaustive search and Rosati's partitioning algorithm, both detailed later [2, Fig. 1 and Thm. 4.15].

E. Proof assistants and Rocq

Proof assistants are software systems useful to formalize mathematical concepts. They are able to verify step by step definitions, theorems, and proofs using their specific language, in order to provide a machine-checked guarantee of the results obtained, relative to the definitions and assumptions stated [6, Core language]. The applicability of formal methods has become evident in academia and industry alike. AWS uses TLA+ specification and model checking for distributed systems [7], and OpenAI has developed a neural theorem prover that finds Lean proofs of formalised olympiad problems [8].

One of the most significant motivations to use proof assistants is that, once a result is checked, it is guaranteed to be correct with respect to the stated definitions and assumptions, as long as the small mathematical kernel inside the system is correct [6, Core language]. In common pen-and-paper proofs, each step has to be verified and may rely on implicit assumptions (think of the circle intersection step in Euclid's construction of the equilateral triangle as a simple example [9, pp. 734–735]).

F. The Rocq proof assistant

The Rocq prover, formerly known as Coq [5], is a mature proof assistant based on the Calculus of Inductive Constructions [6, Core language], used in a variety of scenarios ranging from pure mathematical developments to verified

compilation. CompCert, for example, uses Coq to implement a compiler and prove semantic preservation [10].

It has also been used to teach students and researchers alike about theorem proving, logic, security and programming languages, for example through the *Software Foundations* textbook series [11].

II. REASONING ALGORITHMS

The basic algorithm for reasoning over MKNF and MBNF is the enumeration one. It searches for a countermodel, and answers true if and only if no countermodel is found. The following pseudocode uses MKNF models [1, Sec. 4]; the MBNF version enumerates pairs of an initial world and a non-empty belief cluster [2, Def. 2.1].

```
COUNTERMODEL(T,q):
  for each cluster M
        in powerset(all valuations):
    if M is canonical and
       M is in Gamma(T,M):
      for each I in M:
        if (I,M,M) does not satisfy q:
            return (M,I)
  return none

NAIVE-ENTAILS(T,q):
  return COUNTERMODEL(T,q) is none
```

A second algorithm is the one described by Rosati in Section 4.2, Figure 1, whose correctness is proved in Theorem 4.15 [2]. The pseudocode below includes our query normalisation and removal of repeated modal formulas. The original procedure answers true when entailment fails; ours answers true when it holds.

```
ModalFormulas(T):
    Collect every subformula headed by B
    or not, including nested ones.
    Keep one copy of each distinct formula.

Assignments(A):
    Enumerate every Boolean truth assignment
    to the formulas in A.
    A partition can store just the formulas
    assigned true.

Reduce(f, P):
    If f starts with B or not, return
    the Boolean constant P[f].
    Otherwise preserve atoms and recursively
    reduce Boolean connectives.

ReduceTheory(T, P):
    Return the conjunction of Reduce(f, P)
    for every axiom f in T.

Base(T, P):
    Start with true.
    For every B f in ModalFormulas(T)
    with P[B f] = true:
        conjoin Reduce(f, P).
    Return the resulting objective formula.

Worlds(g):
    Return { i in U : g is true at i }.

SAT(g):
    Enumerate U and return true if
    some world satisfies g.
    Return false after checking every world
    without finding a witness.

Accurate(P, A, M, N):
    Pick any initial world i0 in U.
    For every a in A:
        if P[a] != Eval(a, i0, M, N):
            return false
    return true

QueryReduct(q, M):
    Read the actual truth of each modal
    subformula of q at (i0,M,M).
    Use those values to reduce q
    to an objective formula.
    Include the query's own modal
    subformulas when reading these values.

NormalizeQuery(q):
    Recursively replace each not f
    by ~ B NormalizeQuery(f).
    Recursively preserve all other constructors.
    Query evaluation uses equal belief and
    assumption clusters, where this
    transformation preserves truth.

function RosatiEntails(T, q):
    U := AllBooleanValuations(n)
    A := ModalFormulas(T)
    q := NormalizeQuery(q)

    for each P in Assignments(A):
        M := Worlds(Base(T, P))

        if M is empty:
            continue

        // Condition (a): accuracy
        // of the candidate assignment.
        if not Accurate(P, A, M, M):
            continue

        // Condition (b): a possible initial
        // world refutes the query.
        if not SAT(ReduceTheory(T, P)
            and ~ QueryReduct(q, M)):
            continue

        // Condition (c): minimal belief
        // with fixed assumptions M.
        defeated := false

        for each R in Assignments(A)
            with R != P:
            if not SAT(ReduceTheory(T, R)):
                continue

            L := Worlds(Base(T, R))

            if Accurate(R, A, L, M)
                and M subset L:
                defeated := true
                break

        if not defeated:
            return false

    return true
```

III. A PRACTICAL APPLICATION

In order to demonstrate Rosati's partitioning algorithm in practice, a murder mystery story has been placed after the correctness proofs. The case witnesses a progressive change

in beliefs as the detective uncovers more and more clues, confessions and deductions.

> That had been Mr. Charlie's quiet existence until he was found dead.
> In his own study.
> Poisoned.
> It was Alice who had apparently discovered the body. And, as David explained, the only people in the house the previous day had been Alice and Bob.
> I glanced at David. There were two people to question and a dead man.
> The mystery was ours to unravel.

IV. CONCLUSIONS

This project has formalised two different logics, with several applications in various fields. This has led to three consequences: it certifies (modulo the correctness of Rocq kernel) some relevant theorems and algorithms; it provides correct reasoning algorithms (both naive and more efficient), and shows them in action in a practical scenario. Additionally, it provides a learning path for learning and proving these logics, potentially paving the way to more works of this kind.

REFERENCES

[1] Vladimir Lifschitz. "Nonmonotonic Databases and Epistemic Queries". In: *Proceedings of the Twelfth International Joint Conference on Artificial Intelligence*. 1991, pp. 381–386. URL: https://www.ijcai.org/Proceedings/91-1/Papers/059.pdf.

[2] Riccardo Rosati. "Reasoning about Minimal Belief and Negation as Failure". In: *Journal of Artificial Intelligence Research* 11 (1999), pp. 277–300. DOI: 10.1613/jair.637.

[3] Vadim Ivanov, Matthias Knorr, and João Leite. "Reasoning over Ontologies and Non-monotonic Rules". In: *Progress in Artificial Intelligence*. Vol. 9273. Lecture Notes in Computer Science. Springer, 2015, pp. 388–401. DOI: 10.1007/978-3-319-23485-4_39.

[4] Marco Alberti et al. "Normative Systems Represented as Hybrid Knowledge Bases". In: *Computational Logic in Multi-Agent Systems*. Vol. 6814. Lecture Notes in Computer Science. Springer, 2011, pp. 330–346. DOI: 10.1007/978-3-642-22359-4_23.

[5] The Rocq Development Team. *About the Rocq Prover*. URL: https://rocq-prover.org/about (visited on November 2, 2026).

[6] The Rocq Development Team. *The Rocq Prover Reference Manual*. Version 9.1.0. 2025. URL: https://rocq-prover.org/doc/V9.1.0/refman/language/core/index.html (visited on November 2, 2026).

[7] Chris Newcombe et al. "How Amazon Web Services Uses Formal Methods". In: *Communications of the ACM* 58.4 (2015), pp. 66–73. DOI: 10.1145/2699417.

[8] Stanislas Polu, Jesse Michael Han, and Ilya Sutskever. *Solving (some) Formal Math Olympiad Problems*. OpenAI. February 2, 2022. URL: https://openai.com/index/formal-math/ (visited on November 2, 2026).

[9] Jeremy Avigad, Edward Dean, and John Mumma. "A Formal System for Euclid's Elements". In: *The Review of Symbolic Logic* 2.4 (2009), pp. 700–768. DOI: 10.1017/S1755020309990098.

[10] Xavier Leroy. "Formal Verification of a Realistic Compiler". In: *Communications of the ACM* 52.7 (2009), pp. 107–115. DOI: 10.1145/1538788.1538814.

[11] Benjamin C. Pierce et al. *Logical Foundations*. Ed. by Benjamin C. Pierce. Vol. 1. Software Foundations. Version 7.1. Electronic textbook, 2026. URL: https://softwarefoundations.cis.upenn.edu/lf-current/index.html (visited on November 2, 2026).